МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2

по курсу “Компьютерная графика”

Камера и освещение

Выполнил: Д. А. Широбоков
Группа: М8О-301БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель лабораторной работы: реализовать камеру с возможностью перемещения и освещение сцены по модели Блинна-Фонга с использованием множества типов источников света.

Требования к программе:

• Программа должна работать в реальном времени, с возможностью взаимодействия с камерой (позиция, поворот, угол обзора), источниками освещения (позиция, цвет, интенсивность и параметры, уникальные для соответствующего типа источника), а также параметрами материала (альбедо, цвет блика, коэффициент блика) через пользовательский интерфейс.

• У вершин, помимо позиции, должен быть задан атрибут нормали. Нормали вершин рисуемого объекта генерируются процедурно.

• Для каждого источника света должна быть реализована модель Блинна-Фонга во фрагментном шейдере.

• Для точечного и прожекторного источника нужно реализовать затухание по закону обратных квадратов.

• Для прожекторного источника нужно реализовать мягкие края освещаемой области: для этого задаются параметры внешнего и внутреннего угла.

• Для доступа к массивам памяти для точечных и прожекторных источников света используются storage-буферы, чтобы не задавать максимальное количество элементов массива в шейдере.

Выполненный вариант задания: базовое задание и первые три дополнительных задания (на оценку 4).

1. Разделение внешних данных на два набора дескрипторов: «сцена» и «модель».

2. Освещение по модели Блинна-Фонга массивом точечных источников света из storage-буфера с затуханием по закону обратных квадратов; в интерфейсе реализованы добавление и удаление источников с изменением их параметров.

3. Освещение по модели Блинна-Фонга массивом прожекторных источников света из storage-буфера с затуханием по закону обратных квадратов и мягкими краями освещаемой области; в интерфейсе реализованы добавление и удаление источников с изменением их параметров.

Метод решения

1. Геометрия объекта и процедурные нормали

Используется пирамида из лабораторной работы №1 (вариант №6): 5 вершин в локальной системе координат, центр основания в начале координат, высота 1, основание — квадрат со стороной 1:

Индекс	Позиция (x, y, z)	Назначение
0	(0.5, -0.5, 0.5)	Основание, правый передний
1	(0.5, -0.5, -0.5)	Основание, правый задний
2	(-0.5, -0.5, -0.5)	Основание, левый задний
3	(-0.5, -0.5, 0.5)	Основание, левый передний
4	(0.0, 0.5, 0.0)	Вершина

Нормаль грани вычисляется процедурно на CPU при построении геометрии из положений вершин:

```
vec3 faceNormal(vec3 v0, vec3 v1, vec3 v2) {
return normalize(cross(v0 - v1, v2 - v1));
}
```

Порядок обхода вершин выбран так, чтобы нормали боковых граней смотрели наружу, а нормаль основания — вниз. Дополнительно построены пол (плоскость XZ с нормалью вверх), куб и конус для визуализации источников света.

2. Камера и матрицы вида и проекции

Камера задаётся позицией eye, азимутом yaw, тангажом pitch и углом обзора fov. Вектор направления взгляда и базис камеры:

```
forward = (sin(yaw)·cos(pitch), sin(pitch), −cos(yaw)·cos(pitch))
right = normalize(cross(forward, (0, 1, 0)))
up = cross(right, forward)
```

Матрица вида строится функцией lookAt, матрица проекции — функцией perspective:

```
V = lookAt(eye, eye + forward, up)
P = perspective(fovY, aspect = W/H, near = 0.1, far = 120)
```

Итоговая матрица: viewProjection = P · V.

Отражение оси Y. В Vulkan ось Y в NDC направлена вниз, поэтому изображение отражается по вертикали. Отражение выполняется в матрице проекции (projection[1][1] *= −1), а не в матрице вида. Это принципиально: у проекции в строке Y всего одно ненулевое значение, поэтому знак меняется у всей строки разом. Отражение одного элемента матрицы вида (view[1][1] *= −1) при наклонной камере искажает базис вида (строки перестают быть ортогональными) и ломает winding у части граней: они отсекаются поодиночке, и объект выглядит «съеденным». Поскольку ось Y отражается, в пайплайне задано frontFace = VK_FRONT_FACE_CLOCKWISE.

3. Модель освещения по Блинну-Фонгу

Во фрагментном шейдере для каждого источника вычисляется рассеянная составляющая и блик:

```
blinnPhong(N, V, L, lightColor, albedo, specular, shininess):
nDotL = max(dot(N, L), 0)
H = normalize(L + V)
nDotH = max(dot(N, H), 0)
specular = pow(nDotH, shininess) · nDotL
return lightColor · (albedo · nDotL + specularColor · specular)
```

Рассеянный (ambient) свет задаётся отдельно и умножается на альбедо объекта:

```
color = ambientColor · ambientIntensity · albedo
```

Нормали в шейдере преобразуются в мировые пространства матрицей inverse-transpose, что корректно учитывает неравномерный масштаб модели:

```
normalMatrix = transpose(inverse(mat3(model)))
```

4. Затухание и мягкие края конуса

Для точечного и прожекторного источников реализовано затухание по закону обратных квадратов с мягким срезом у границы радиуса действия:

attenuation(d, range):
$d^2 = \max(d \cdot d, 1e{-}4)$
ratio = clamp(d / range, 0, 1)
cutoff = clamp($1 - \text{ratio}^4$, 0, 1)
return cutoff / d^2

Прожекторный источник дополнительно ограничен конусом. Мягкий край реализован через плавный переход smoothstep между косинусом внешнего угла (полное освещение начинается) и косинусом внутреннего угла (освещение достигает максимума):

cosAngle = dot(−L, direction)
spot = smoothstep(cosOuter, cosInner, cosAngle)

Если внешний угол шире внутреннего, то cosOuter < cosInner, и интенсивность плавно нарастает от края конуса к его оси. Итоговый вклад источника умножается на attenuation и spot.

5. Два набора дескрипторов: «сцена» и «модель»

Внешние данные разделены на два набора дескрипторов согласно дополнительному заданию 1.

Набор	Содержимое	Тип ресурса
set 0 — сцена	viewProjection, позиция камеры, ambient, направленный источник, счётчики источников	uniform buffer (144 байт)
set 0 — сцена	Массив точечных источников: positionRange, colorIntensity	storage buffer (32 байт на элемент)
set 0 — сцена	Массив прожекторных источников: positionRange, directionCosOuter, colorIntensity, innerCos	storage buffer (64 байта на элемент)
set 1 — модель	Матрица трансформации, матрица нормалей, альбедо, цвет и коэффициент блика	uniform buffer dynamic (160 байт)

Число элементов в массивах источников не задаётся константой в шейдере: размер storage-буфера определяется данными, а сам счётчик передаётся в uniform-блоке сцены (lightCounts). Это позволяет добавлять

и удалять источники без перекомпиляции шейдеров. При росте числа источников storage-буфер пересоздаётся, а дескриптор обновляется через vkUpdateDescriptorSets.

Каждому объекту соответствует свой слот в общем буфере моделей: доступ к слоту задаётся динамическим смещением в дескрипторе (VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC). Шаг слота вычисляется по ограничению minUniformBufferOffsetAlignment, а диапазон дескриптора равен размеру одного блока, чтобы смещение не выходило за границы буфера.

6. Объекты Vulkan, использованные в работе

Объект	Назначение
VkInstance, VkPhysicalDevice, VkDevice	Инициализация через vk-bootstrap (validation layers, preference discrete GPU)
VkSurfaceKHR (GLFW)	Окно для презентации
VkSwapchainKHR (FIFO)	Буферизация, VK_PRESENT_MODE_FIFO_KHR
VkRenderPass (2 attachment)	Цветной (swapchain image) + глубинный (D24_UNORM_S8_UINT)
VkDescriptorSetLayout, VkDescriptorPool, VkDescriptorSet	Два набора дескрипторов: сцена (uniform + 2 storage) и модель (uniform dynamic)
VkPipelineLayout + VkPipeline	Графический конвейер: vertex + fragment, TRIANGLE_LIST, back-face culling, FRONT_FACE_CLOCKWISE, depth test LESS
VkBuffer (vertex) + VMA	Геометрические данные и буферы сцены (host-visible, HOST_COHERENT)
VkCommandPool + VkCommandBuffer	Запись команд рендера сцены и ImGUI
VkSemaphore / VkFence	Синхронизация: image_available → render → image_finished → present
ImGui Vulkan backend	Отдельный рендер-пасс и дескрипторный пул, отрисовка UI поверх сцены

7. Организация шейдеров

Используются четыре шейдера:

• scene.vert / scene.frag — освещаемые объекты: чтение блоков сцены и модели, расчёт освещения по Блинну-Фонга;

• unlit.vert / unlit.frag — визуализация источников света: постоянный цвет, не освещаемый по модели Блинна-Фонга. Для прозрачного конуса используется отдельный пайплайн с альфа-смешиванием и отключённой записью в буфер глубины.

8. Взаимодействие с пользователем

Элемент	Реализация
Вращение камеры мышью	GLFW_MOUSE_BUTTON_LEFT + glfwSetCursorPosCallback; дельта курсора → yaw и pitch (pitch ограничен ±89°). Ввод игнорируется, если ImGui::GetIO().WantCaptureMouse.
Угол обзора	Колесо мыши (glfwSetScrollCallback) изменяет fov в диапазоне 15°…90°
Перемещение камеры	Клавиши W/A/S/D — движение по направлению взгляда и вбок, Q/E — вверх и вниз
Панель «Камера»	DragFloat3 для позиции, DragFloat для азимута, тангажа и угла обзора
Панель «Сцена»	Цвет и сила ambient, параметры направленного источника, материалы пирамиды и пола (альбедо, цвет блика, коэффициент блика), трансформация объекта и параметры анимации
Панель «Источники света»	Добавление и удаление точечных и прожекторных источников, изменение позиции, цвета, интенсивности, радиуса действия, углов прожектора, прикрепление источника к пирамиде, показ/скрытие визуализации

9. Визуализация источников света

Для точечного источника рисуется яркий куб цвета источника, для прожекторного — куб и полупрозрачный конус, показывающий направление и угол луча. Эти фигуры имеют постоянный цвет и не освещаются по модели Блинна-Фонга: они рисуются отдельным пайплайном (unlit) с шейдером, передающим цвет напрямую из альбедо модели.

Результаты

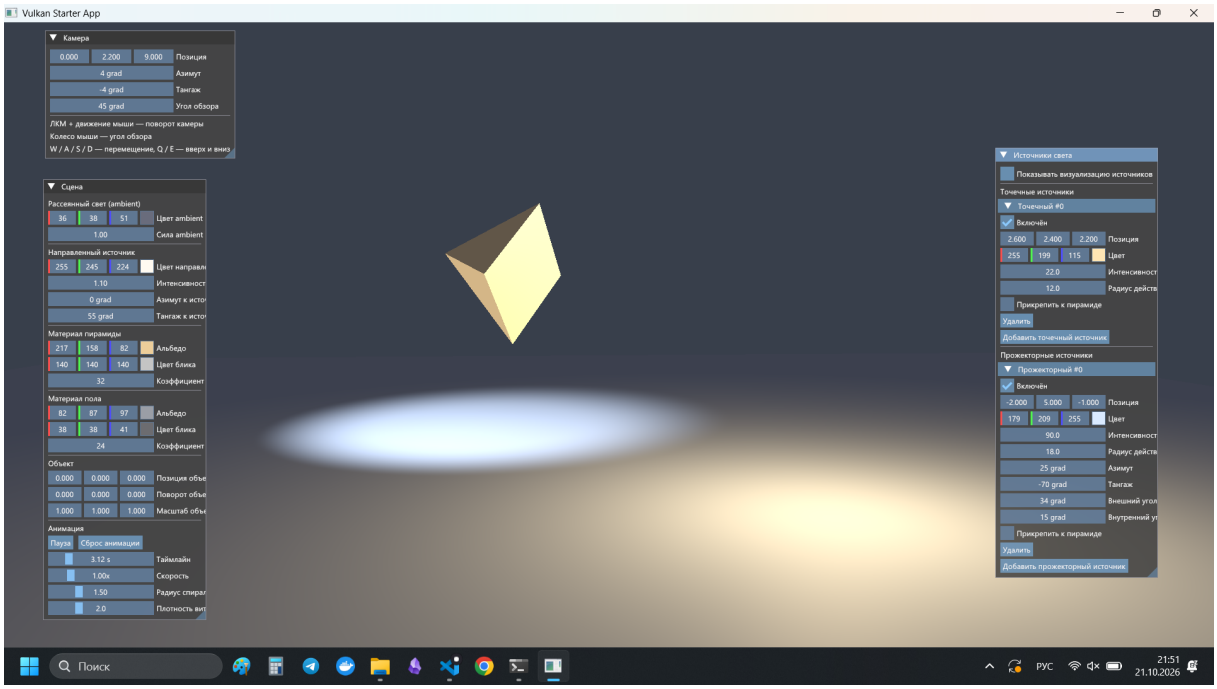


Рисунок 1 — Общий вид сцены: пирамида с процедурно вычисленными нормалями освещена направленным и прожекторным источниками света, на полу видно пятно прожектора с мягкими краями. Слева — панели «Камера» и «Сцена», справа — панель «Источники света» с параметрами точечного и прожекторного источников.

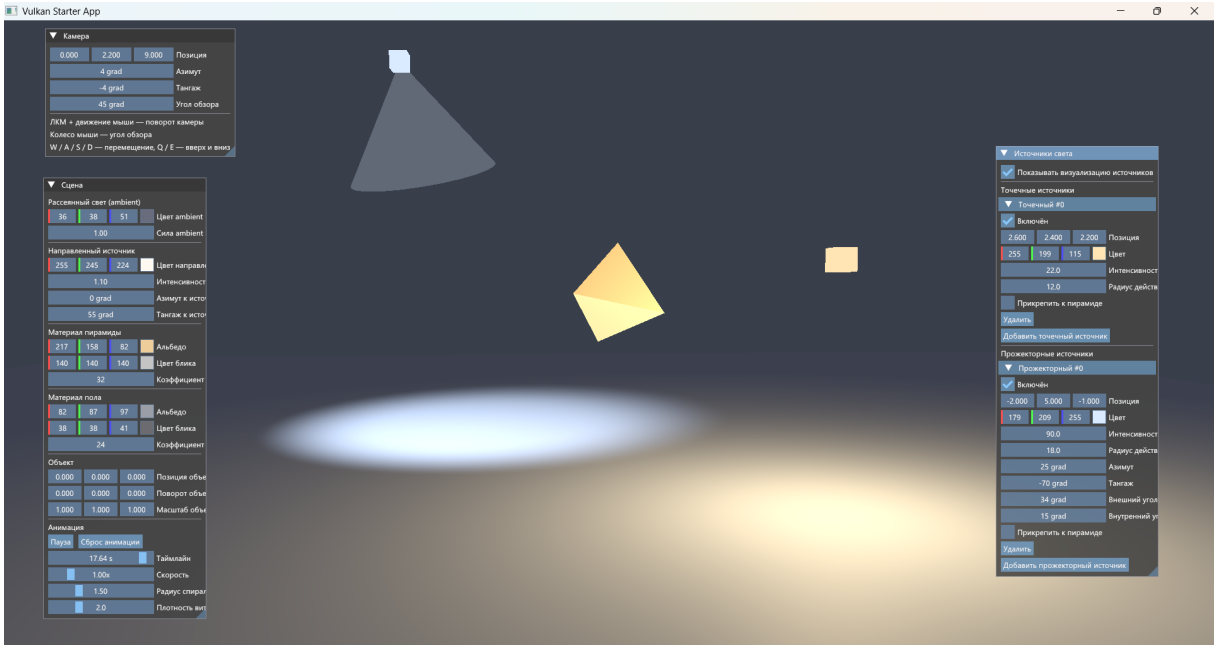


Рисунок 2 — Визуализация источников света: яркий куб цвета точечного источника и полупрозрачный конус, показывающий направление и угол луча прожекторного источника. Фигуры имеют постоянный цвет и не освещаются по модели Блинна-Фонга. Камера повернута (азимут −4°, тангаж −4°), поэтому видна пирамида с другой стороны и пятно прожектора на полу.

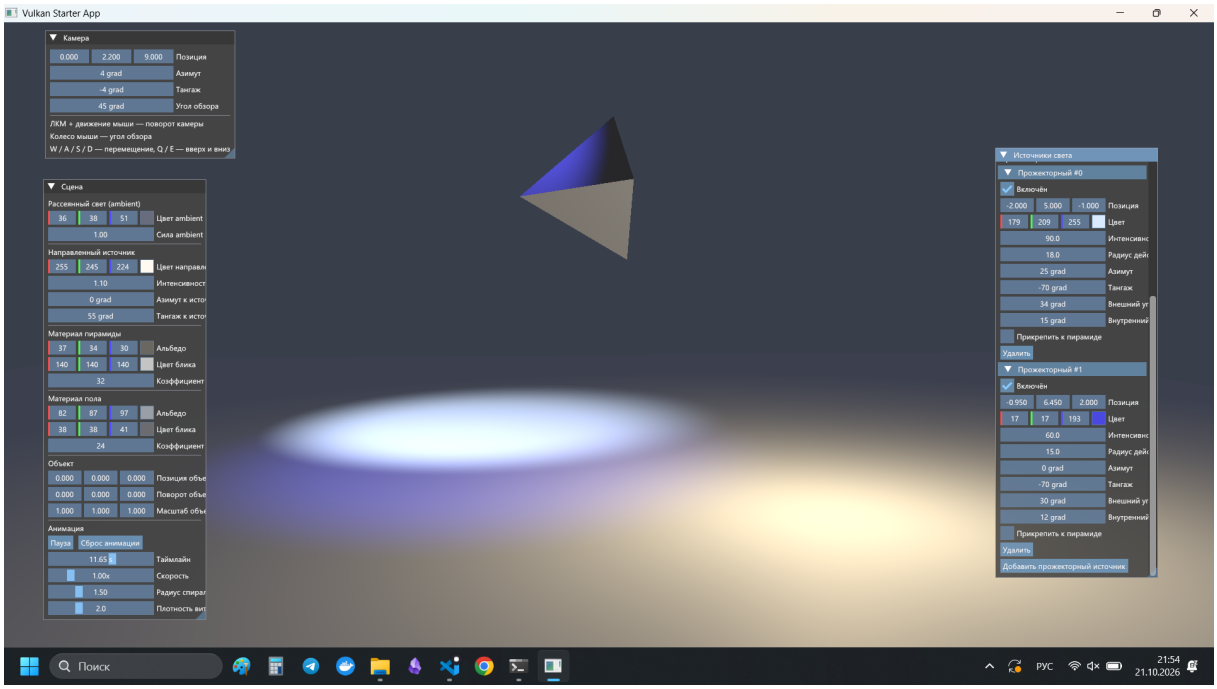


Рисунок 3 — Изменение параметров через интерфейс в реальном времени: добавлен второй прожекторный источник (синий, интенсивность 60, радиус действия 15), изменено альбедо пирамиды, в результате чего её освещение стало синим, а на полу видно пятна от двух прожекторов. В панели «Источники света» видны оба прожекторных источника с их параметрами.

Выводы

В ходе работы я научился(-ась):

1. Реализовывать интерактивную камеру в Vulkan: матрицы вида и проекции, управление положением, поворотом и углом обзора, раздельную обработку ввода для сцены и интерфейса ImGUI.

2. Применять модель освещения Блинна-Фонга для нескольких типов источников света (направленного, точечного, прожекторного) непосредственно во фрагментном шейдере, включая затухание по закону обратных квадратов и мягкие края конуса прожектора.

3. Организовывать передачу данных в шейдеры через дескрипторы: разделять данные на два набора («сцена» и «модель»), использовать storage-буферы для массивов источников света и динамические смещения для доступа к параметрам отдельных объектов.